



सो, दिस वीडियो इज़ ऑल अ...
TXT 309.75KB

make notes for me out of this transcript , explain everything properly deeply

Numpy (NPI) — Complete Notes (from the Transcript)

Below are deep, structured notes covering **everything** in the transcript: notebooks, Anaconda, NumPy origin, array creation, indexing, operations, exercises, and more.

1. Introduction

- This video is all about **NPI (NumPy)** — a **Python library**.
- Topics covered:
 - How NumPy arrays are created
 - How NumPy operations are performed
 - Projects/exercises
- Prerequisite: **Python** (watch the previous Python video first).
- Also learn: **Jupyter Notebooks** — how to create and work with them.

2. What Are Notebooks?

- A **notebook** is a special file with extension `.ipynb` (IPython Notebook).
- Python file extension is `.py` ; notebook is `.ipynb` .
- It lets you:
 - Write code
 - Run code
 - See output
 - Add notes, equations, images, charts
 - Everything in one place — great for **data visualization** and **data science**.

Why Notebooks for Data Science?

- Data science involves ML, AI, data analysis, visualization.
- Notebooks make it easy to combine code + charts + notes.
- In plain `.py` files, charts/graphs are harder to manage.

3. How to Download Notebooks

3.1 Using Anaconda (Recommended for Beginners)

What is Anaconda?

- A **distribution** — a bundle of pre-installed Python libraries (NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn, TensorFlow, PyTorch, etc.).
- Normally, each library must be installed manually via `pip install`.
- Anaconda comes with everything pre-installed.

Steps:

1. Search "**Anaconda**" on Google → go to download page.
2. Enter email → submit → get **Download Now** page.
3. Download for your OS (Windows/Mac/Linux).
4. File is large (contains all packages).
5. Install — **tick all options** during installation (warnings are fine).
6. Installation takes time.

Opening Jupyter Notebook via Anaconda:

1. Open **Anaconda Navigator** (from Applications).
2. Click **Launch** on **Jupyter Notebook**.
3. A terminal opens → browser opens with Jupyter interface.
4. You'll see many folders (Anaconda Projects, Desktop, Documents).
5. To open your folder:
 - Click **New** → **Terminal**.
 - Use `cd` (change directory) to navigate:

```
text
```

```
cd Desktop/NumPy_YouTube
```

- Then run:

```
text
```

```
jupyter notebook
```

6. Your folder opens in Jupyter.

7. Click **New** → **Python 3 (ipykernel)** → creates a new notebook.

Shortcut (may not work for everyone):

- Right-click your folder → **Open Terminal** → type `jupyter notebook` .

Troubleshooting:

- If errors, run **Anaconda Navigator as Administrator**.

3.2 Using VS Code

1. Open your folder in VS Code.
2. Install the **Jupyter extension** (Extensions → search "Jupyter" → install).
3. Open/create a `.ipynb` file.
4. Select a **kernel** (Python environment):
 - Choose **Global** or **Conda environment**.
 - If using global, you may need `pip install numpy` etc. manually.
5. VS Code will suggest required installs — accept them.
6. If using Anaconda environment, all libraries are available.

Note: Without Anaconda, you must install each library individually (`pip install numpy` , `pip install pandas` , etc.). Developers often use a **requirements.txt** file for this.

3.3 Using Google Colab (Best for Weak Systems)

- Go to **Google Colab**.
- Click **New Notebook**.

- Syncs with Google Drive.
- **Pros:**
 - Free **GPU support** (great for ML/DL later).
 - All libraries pre-installed.
 - No setup needed.
- **Cons:**
 - Runs on cloud → needs fast internet.
 - Slightly slower execution if network is slow.

4. Jupyter Notebook Interface & Settings

- **Cell** = basic unit. Write code, press **Shift+Enter** to execute and move to next cell.
- Multiple cells are **interconnected** — variables/functions defined in one cell are available in others.
- Example:
 - Cell 1: `a = 12`
 - Cell 2: `print("Hello")` → Shift+Enter → prints Hello.
 - Cell 3: define function `def hello(): print("Hello how are you?")` → run.
 - Cell 4: call `hello()` → works.

Recommended Settings:

- **Settings** → **Themes** → **Jupyter Dark** (easier on eyes).
- **Settings** → **Editor** → **Enable Auto Complete** (essential — NumPy has thousands of functions).
- **Settings** → **Editor** → **Auto Close Brackets** (enable).

5. Origin Story of NumPy

- Python was always powerful with math, but **missing arrays**.
- Python had lists, tuples, dictionaries, sets — but no arrays.

- **1995:** Jim Hugunin created **Numeric library** — provided array support, but performance was poor.
- **Early 2000s:** Space Telescope Science Institute developed **Numarray** — better, but not compatible with Numeric.
- **2005:** Travis Oliphant combined **Numeric + Numarray** → created **NumPy**.
- Finally, Python had proper arrays.

Why NumPy is Fast:

- Written in **C** (compiled language) → super fast.
- Python is **interpreted** → slow.
- NumPy arrays are up to **50x faster** than Python lists.

6. NumPy Arrays vs Python Lists

Feature	Python List	NumPy Array
Data Types	Heterogeneous (mixed types allowed)	Homogeneous (same type; mixed types get converted)
Performance	Slow (interpreted)	Fast (C-compiled, up to 50x faster)
Memory Efficiency	Low (more space)	High (less space)
Vector Operations	Manual loops needed	Broadcasting & vectorized operations

Data Type Conversion in NumPy:

- If you pass `[1, 2, 3.5, "hello"]` to `np.array()`, all become **strings** (U32).
- If no string but a float exists, all become **floats**.

7. Creating NumPy Arrays

7.1 Basic Array Creation

python

```
import numpy as np

arr = np.array([1, 2, 3, 4])
print(arr)  # [1 2 3 4]
```

- Created from a **list**.
- Prints like a list but without commas (and with `array` prefix if not printed directly).

7.2 Vectors vs Matrices vs Tensors

- **Vector** = 1D array (single row).
- **Matrix** = 2D array (rows × columns).
- **Tensor** = 3D, 4D, ... (higher dimensions).

Creating a 2D Matrix:

python

```
arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(arr)
# [[1 2 3]
#   [4 5 6]
#   [7 8 9]]
```

- Python lists don't visually support multi-dimensional structure well.
- NumPy arrays show the 2D structure properly.

8. Array Generation Functions

8.1 `np.arange()`

- Works like `range()` — start, stop, step.
- **Stop is exclusive.**

python

```
arr = np.arange(1, 11)      # 1 to 10
arr = np.arange(1, 11, 2)   # 1, 3, 5, 7, 9
```

8.2 np.zeros()

python

```
np.zeros(6)      # [0. 0. 0. 0. 0. 0.]
np.zeros((4, 8)) # 4 rows, 8 columns of zeros
```

8.3 np.ones()

python

```
np.ones(6)      # [1. 1. 1. 1. 1. 1.]
np.ones((6, 6)) # 6x6 matrix of ones
```

8.4 np.linspace()

- Returns **evenly spaced numbers** over an interval.
- `np.linspace(start, stop, num)` — `num` = total elements.

python

```
np.linspace(1, 5, 2)    # [1. 5.]
np.linspace(1, 5, 3)    # [1. 3. 5.]
np.linspace(1, 5, 5)    # [1. 2. 3. 4. 5.]
np.linspace(0, 1, 100)  # 100 evenly spaced numbers between 0 and 1
```

Difference from `arange` : `arange` uses step size; `linspace` uses number of elements.

9. Random Number Generation

9.1 np.random.rand()

- Returns random numbers between **0 and 1** (normalization).
- `np.random.rand(5)` → 5 random numbers (vector).

- `np.random.rand(5, 5)` → 5x5 matrix.

9.2 `np.random.randn()`

- Returns random numbers from **standard normal distribution** (mean 0, variance 1).
- Values typically between **-3 and +3** (standardization).

9.3 `np.random.randint()`

- Returns random **integers**.
- `np.random.randint(6)` → one random integer from 0 to 5.
- `np.random.randint(10, 20, 10)` → 10 random integers between 10 and 19.

Concepts:

- **Normalization:** values between 0 and 1.
- **Standardization:** values between -3 and +3.

10. Array Attributes

Attribute	Description	Example
<code>arr.shape</code>	Rows × Columns	(3, 3)
<code>arr.size</code>	Total elements	9
<code>arr.dtype</code>	Data type	int64

- **Attributes:** no brackets `()`.
- **Methods:** use brackets `()`.

11. Array Methods

python

```
arr.min()    # Minimum element
arr.max()    # Maximum element
```

```
arr.sum()    # Sum of all elements
arr.mean()   # Average
arr.std()    # Standard deviation
arr.argmax() # Index of max element
arr.argmin() # Index of min element
```

Axis in `np.sum()` :

- `np.sum(arr, axis=0)` → **column-wise** sum.
- `np.sum(arr, axis=1)` → **row-wise** sum.

Memory trick:

- Axis 0 = vertical (columns combined).
- Axis 1 = horizontal (rows combined).

12. Reshaping & Resizing

12.1 `reshape()`

- Changes shape **without changing data**.
- Total elements must match.

python

```
arr = np.arange(1, 31)          # 30 elements
arr.reshape(6, 5)               # 6 rows, 5 columns
arr.reshape(5, 6)               # 5 rows, 6 columns
arr.reshape(6, 5)               # Works
arr.reshape(5, 5)               # ERROR (25 ≠ 30)
```

12.2 `resize()`

- Changes the array **in-place**.
- Subsequent references use the new shape.

13. Indexing & Slicing

13.1 Vector Indexing

python

```
arr = np.arange(11, 21)
arr[6]    # 17 (0-indexed)
arr[9]    # 20
```

13.2 Vector Slicing

python

```
arr[1:5]    # indices 1,2,3,4
arr[:5]     # 0 to 4
arr[3:]     # 3 to end
arr[3:9:2]  # step of 2
```

Rule: Last index is **exclusive**.

13.3 Matrix Indexing

python

```
arr = np.arange(1, 31).reshape(6, 5)
arr[0]      # First row
arr[5]      # Last row
arr[0, 0]   # Element at row 0, col 0 → 1
arr[0, 4]   # Element at row 0, col 4 → 5
arr[5, 4]   # Element at row 5, col 4 → 30
```

Format: arr[row, column]

13.4 Matrix Slicing

python

```
arr[0:2, 1:3]  # Rows 0-1, Columns 1-2
arr[3:, 3:]    # Rows 3-end, Columns 3-end
arr[:, 2]      # Entire column 2 (returns a vector)
```

13.5 Boolean Indexing

python

```
arr = np.arange(11, 21)
bool_idx = arr % 2 == 0
arr[bool_idx]    # [12, 14, 16, 18, 20]
```

- Boolean array must have **same length** as original array.
- `True` values are kept; `False` values are removed.

14. Arithmetic Operations

python

```
a1 = np.array([1, 2, 3, 4, 5])
a2 = np.array([6, 7, 8, 9, 10])

a1 + a2    # [7, 9, 11, 13, 15]
a1 - a2    # [-5, -5, -5, -5, -5]
a1 * a2    # [6, 14, 24, 36, 50]
a1 / a2    # [0.166..., 0.285..., ...] → boolean if compared
a1 // a2   # [0, 0, 0, 0, 0]
a1 ** a2   # [1, 128, 6561, ...]
```

Rule: Arrays must have the **same shape** for element-wise operations.

15. Broadcasting

- Performing operations between arrays of **different shapes**.
- NumPy "broadcasts" the smaller array across the larger one.

python

```
arr = np.array([10, 20, 30, 40])
arr + 10    # [20, 30, 40, 50]
```

- Works for matrices too:

python

```
arr2 = np.arange(1, 26).reshape(5, 5)
arr2 + 10 # Adds 10 to every element
```

16. Deep Copy vs Shallow Copy

Shallow Copy (Slicing)

- When you **slice** an array, the slice is a **view** (shares memory).
- Changes to the slice **do NOT affect** the original array (in NumPy).
- Actually, in NumPy, slicing creates a **view** — changes DO affect original. Wait — the transcript says slicing creates a separate space (deep copy), but then corrects itself.

Transcript's Final Clarification:

- **Shallow copy:** $b = a$ — both point to same memory. Changes in b affect a .
- **Deep copy:** $b = a.copy()$ — separate memory. Changes in b do NOT affect a .
- **Slicing:** In NumPy, slicing creates a **view** — but the transcript demonstrates that changes to a slice do NOT affect the original (this is actually the behavior of `.copy()`). The transcript gets confused but ultimately shows:
 - $b = a \rightarrow$ changes affect both.
 - $b = a.copy() \rightarrow$ changes affect only b .
 - Slicing in NumPy actually creates a **view**, but the transcript shows it as a separate copy. **In real NumPy, slicing creates a view (changes affect original).** The transcript may have made an error here.

Correct NumPy Behavior:

- $b = a \rightarrow$ same object.
- $b = a.copy() \rightarrow$ deep copy.
- $b = a[1:5] \rightarrow$ view (shallow copy — changes affect original).

17. Matrix Operations

17.1 Element-wise Multiplication

python

```
a * b    # Element-wise (Hadamard product)
```

17.2 Matrix Multiplication (Dot Product)

python

```
a @ b          # Matrix multiplication
np.dot(a, b)    # Same
np.matmul(a, b) # Same
```

Rule: Columns of first matrix must equal rows of second matrix.

17.3 Transpose

python

```
a.T    # Rows become columns, columns become rows
```

18. Advanced Array Manipulation

18.1 Stacking

- Only works with vectors (1D arrays), not matrices.

python

```
a = np.array([1, 2, 3, 4])
b = np.array([5, 6, 7, 8])

np.vstack((a, b))    # Vertical stack
np.hstack((a, b))    # Horizontal stack
np.column_stack((a, b)) # Columns
```

18.2 Splitting

python

```
c = np.arange(1, 17).reshape(4, 4)

np.hsplit(c, 2)    # Horizontal split into 2 parts
np.vsplit(c, 2)    # Vertical split into 2 parts
np.split(c, 4)     # Split into 4 parts
```

Rule: Must split into **equal divisions**.

19. Exercises

19.1 Valid Sudoku

Rules:

- 9×9 grid.
- Each row sum = 45.
- Each column sum = 45.
- Each 3×3 block sum = 45.

Solution Approach:

python

```
import numpy as np
s = np.array([...]) # 9x9 Sudoku

# Row sums
row_sums = np.sum(s, axis=1)
# Column sums
col_sums = np.sum(s, axis=0)

# 3x3 block sums
for i in range(0, 9, 3):
    for j in range(0, 9, 3):
        block = s[i:i+3, j:j+3]
        print(block.sum())
```

Validation: All sums must equal 45.

19.2 Student Marks Array

python

```
data = np.array([
    [18, 85, 88],
    [19, 92, 95],
    [17, 76, 81],
    [18, 65, 70],
    [19, 90, 85]
])
# Columns: Age, Maths, Science
```

Questions & Solutions:

1. **Shape:** `data.shape` → (5, 3)
2. **Average age:** `np.mean(data[:, 0])` → 18.4
3. **Maths marks of all students:** `data[:, 1]` → [85, 92, 76, 65, 90]
4. **Highest science marks:** `np.max(data[:, 2])` → 95
5. **Students scoring > 90 in Maths:**

python

```
data[data[:, 1] > 90]
```

→ Returns student with 92 marks.

6. **Increase Maths marks by 5:**

python

```
data[:, 1] += 5
```

7. **Students younger than 19:**

python

```
data[data[:, 0] < 19]
```

→ 3 students.

8. **Count students younger than 19:**

python

```
len(data[data[:, 0] < 19])
```

→ 3

9. Average marks in each subject (column-wise mean):

python

```
np.mean(data[:, 1:], axis=0)
```

→ [86, 83.2] (Maths, Science)

10. Students scoring at least 80 in both subjects:

python

```
data[(data[:, 1] >= 80) & (data[:, 2] >= 80)]
```

→ 3 students.

11. Replace Science marks < 75 with 0:

python

```
data[:, 2][data[:, 2] < 75] = 0
```

20. Key Takeaways

- **NumPy** is a Python library for arrays — faster and more memory-efficient than lists.
- **Notebooks** (.ipynb) are essential for data science — combine code, notes, charts.
- **Anaconda** pre-installs all data science libraries.
- **Google Colab** is best for weak systems + GPU support.
- **Array creation:** `np.array()` , `np.arange()` , `np.zeros()` , `np.ones()` , `np.linspace()` , `np.random.*`
- **Attributes:** `shape` , `size` , `dtype`
- **Methods:** `min()` , `max()` , `sum()` , `mean()` , `std()` , `argmax()` , `argmin()`
- **Indexing/Slicing:** Works like Python lists but multi-dimensional.

- **Boolean Indexing:** Filter arrays with conditions.
- **Broadcasting:** Operations between different-shaped arrays.
- **Deep vs Shallow Copy:** `b = a` (shallow), `b = a.copy()` (deep).
- **Matrix Operations:** `@` for multiplication, `.T` for transpose.
- **Stacking:** `vstack`, `hstack`, `column_stack`.
- **Splitting:** `hsplit`, `vsplit`, `split`.

21. Next Steps

- Next video: **Pandas** (DataFrames).
- All notes available on the **GitHub repository**.
- Practice exercises — don't skip them!

Hashtag: #TillTheEnd



शेरियन सीआई स्कूल ने वादा ...
TXT 383.38KB

make notes for me out of this transcript , explain everything properly
deeply

Pandas — Complete Notes (from the Transcript)

Below are deep, structured notes covering **everything** in the transcript: Pandas origin, Series, DataFrames, indexing, missing data, merging, grouping, pivot tables, operations, feature extraction, projects, and more.

1. Introduction

- **Pandas** is a Python library for data manipulation and analysis.
- Prerequisites: **Python** and **NumPy** (watch previous videos first).
- The video covers:

- Theory of Pandas
- How functions are applied
- Real-life projects (Capstone)
- All notebooks (.ipynb) are available on the **GitHub repository**.

Quote: *"In the world of data science, mastering Pandas is like learning to read before writing poetry."*

2. Origin Story of Pandas

- **2008:** American data scientist **Wes McKinney** started developing Pandas while working at a financial company.
- He needed a Python library for data analysis that was **fast, flexible, and easy to use**.
- Python lacked tools to efficiently manage and analyze large datasets.
- **Name Origin:** Not named after the animal. It comes from **"Panel Data"** — an econometrics term for large multidimensional datasets.
- Today, Pandas is the most basic and important tool for ML and data science.

3. Use Cases of Pandas

- Data Cleaning
- Data Analysis
- Data Transformation
- Data Visualization
- Data Aggregation
- File Handling
- Data Filtering & Selection
- Time Series Analysis

Key takeaway: The word "Data" in Data Science depends heavily on Pandas. It's a must-learn.

4. What You Will Learn

- **Series** (1D labeled array)
- **DataFrames** (2D table — most important)
- Missing Data
- Merging, Joining, Concatenation
- Group By & Aggregation
- Pivot Tables & Cross Tabs
- Operations
- Working with Real Data
- Input/Output
- Capstone Projects

5. Pandas Series

5.1 What is a Series?

- A **1D labeled array** (vertical).
- Can hold **any data type** (unlike NumPy arrays which are homogeneous).
- Index labels are collectively called the **index**.
- Looks vertical (unlike NumPy arrays which look horizontal).

5.2 Creating a Series

Import:

```
python
```

```
import numpy as np
import pandas as pd
```

From a List:

```
python
```

```
labels = ['A', 'B', 'C']  
my_list = [10, 20, 30]  
pd.Series(data=my_list, index=labels)
```

Output:

text

```
A    10  
B    20  
C    30
```

From a NumPy Array:

python

```
arr = np.array([10, 20, 30])  
pd.Series(arr)
```

From a Dictionary:

python

```
d = {1: 10, 2: 20, 3: 30}  
pd.Series(d)
```

- Keys become **index labels** automatically.
- Values become the data.

Important: Series is **1D only**. Passing a 2D array will cause an error.

6. Pandas DataFrame

6.1 What is a DataFrame?

- When you **combine multiple Series**, you get a **DataFrame**.
- It's a 2D table with **rows and columns**.
- Most important structure in Pandas.

6.2 Creating a DataFrame

From a Dictionary:

python

```
data = {  
    'Name': ['John', 'Anna', 'Peter', 'Linda'],  
    'Age': [28, 34, 29, 42],  
    'City': ['New York', 'Paris', 'Berlin', 'London'],  
    'Salary': [65000, 70000, 62000, 85000]  
}  
df = pd.DataFrame(data)
```

From a List:

python

```
data_list = [  
    ['John', 28, 'New York', 65000],  
    ['Anna', 34, 'Paris', 70000],  
    ['Peter', 29, 'Berlin', 62000],  
    ['Linda', 42, 'London', 85000]  
]  
df2 = pd.DataFrame(data_list, columns=['Name', 'Age', 'City', 'Salary'])
```

- Without `columns`, default column names are `0`, `1`, `2`, `3`.

6.3 Selecting & Indexing Columns

Select one column:

python

```
df['Name'] # Returns a Series
```

Select multiple columns:

python

```
df[['Name', 'City']] # Returns a DataFrame
```

Create a new column:

python

```
df['Designation'] = ['Doctor', 'Engineer', 'Doctor', 'Engineer']
```

- Must provide exactly as many values as rows.

6.4 Removing Columns

python

```
df.drop('Designation', axis=1)
```

- `axis=1` → columns (vertical drop)
- `axis=0` → rows (horizontal drop)
- **Default axis is 0.**

In-place removal:

python

```
df.drop('Designation', axis=1, inplace=True)
```

- Without `inplace=True`, the original DataFrame is unchanged.

Drop multiple columns:

python

```
df.drop(['City', 'Salary'], axis=1, inplace=True)
```

Drop rows:

python

```
df.drop(0, axis=0) # Drops first row
```

7. Selecting Rows

7.1 Using `.loc[]` (Location)

python

```
df.loc[0]          # Selects row with index 0
df.loc[[0, 1]]     # Selects rows 0 and 1
```

7.2 Using .iloc[] (Index Location)

python

```
df.iloc[0]         # Same as .loc[0] when index is default
```

7.3 Selecting Subsets (Rows + Columns)

python

```
df.loc[[0, 1], ['City', 'Salary']]
```

- First list → rows
- Second list → columns

Example:

python

```
df.loc[2:3, ['Name', 'Age']]
```

8. Conditional Selection

8.1 Single Condition

python

```
df[df['Age'] > 30]
```

8.2 Multiple Conditions (AND)

python

```
df[(df['Age'] > 30) & (df['City'] == 'Paris')]
```

- Use & for AND, | for OR.
- Wrap each condition in parentheses.

8.3 Multiple Conditions (OR)

python

```
df[(df['Age'] > 30) | (df['City'] == 'Paris')]
```

9. Missing Data

9.1 Finding Missing Data

isna() / isnull() :

python

```
df.isna()           # Boolean DataFrame (True where NaN)
df.isna().sum()     # Count of NaN per column
df.isna().any()     # True if any NaN in column
```

9.2 Removing Missing Data

python

```
df.dropna()         # Drops rows with ANY NaN
df.dropna(thresh=3) # Keep rows with at least 3 non-NaN values
df.dropna(axis=1)   # Drops columns with NaN
```

9.3 Filling Missing Data

Fill with a constant:

python

```
df.fillna(0)
df.fillna(0, inplace=True)
```

Fill with different values per column:

python

```
df.fillna({'A': 0, 'B': 100, 'C': 300, 'D': 400})
```

Fill with mean:

python

```
df.fillna(df.mean())
```

Important: Use `inplace=True` to apply changes permanently.

10. Merging, Joining & Concatenation

10.1 Merging (`pd.merge`)

- Combines two DataFrames based on a **common column** (like SQL JOIN).

python

```
pd.merge(employees, salaries, on='EmployeeID')
```

Types of Merge (`how` parameter):

Type	Description
inner (default)	Only matching rows
outer	All rows from both; NaN where no match
left	All rows from left; NaN where no match on right
right	All rows from right; NaN where no match on left

Example:

python

```
pd.merge(employees, salaries, on='EmployeeID', how='outer')
```

10.2 Concatenation (`pd.concat`)

- Stacks DataFrames **vertically** or **horizontally**.

python

```
pd.concat([df1, df2])          # Vertical (axis=0, default)
pd.concat([df1, df2], axis=1)  # Horizontal (axis=1)
```

- `axis=0` → rows (stack on top of each other)
- `axis=1` → columns (side by side)

10.3 Joining (`df.join`)

- Joins on **index** (not columns).

python

```
df1.join(df2)
df1.join(df2, how='outer')
df1.join(df2, how='left')
df1.join(df2, how='right')
```

Difference:

- `merge` → based on columns
- `join` → based on index
- `concat` → just stacking

11. Group By & Aggregation

11.1 Group By

- Groups data by a **categorical column** and performs operations on each group.

python

```
df.groupby('Category')['Sales'].sum()
```

Multiple columns:

python

```
df.groupby(['Category', 'Store'])['Sales'].sum()
```

- **Qualitative (Categorical):** Category, Store, Gender, etc.
- **Quantitative (Numerical):** Sales, Quantity, Price, etc.
- Group by is usually done on **categorical** columns.

11.2 Aggregation Functions

python

```
df['Sales'].mean()  
df['Sales'].median()  
df['Sales'].mode()  
df['Sales'].min()  
df['Sales'].max()  
df['Sales'].count()  
df['Sales'].std()  
df['Sales'].var()
```

Multiple aggregations at once:

python

```
df['Sales'].agg(['mean', 'median', 'min', 'max', 'count', 'std'])
```

- **Note:** `mode` may cause errors in some Pandas versions.

12. Pivot Tables

12.1 What is a Pivot Table?

- Reshapes data — creates a new DataFrame with:
 - **Index** (rows)
 - **Columns**
 - **Values** (aggregated)

python

```
pd.pivot_table(df, values='Sales', index='Region', columns='Product', aggfunc='mean')
```

- Default aggregation is **mean**.
- Can change to `median`, `sum`, `count`, etc.

Example:

python

```
pd.pivot_table(df, values=['Sales', 'Units'], index='Region', columns='Product', aggfunc='median')
```

Rules:

- **Values** must be **numeric** (for aggregation).
- **Index** and **Columns** can be categorical.
- Used for **heatmaps** and **clustered visualizations**.

12.2 Cross Tabs (`pd.crosstab`)

- Similar to Pivot Table but used for **counting**.

python

```
pd.crosstab(df['Region'], df['Product'])
```

- Counts occurrences instead of aggregating.
- Used for **frequency tables**.

Difference:

- **Pivot Table** → aggregation (mean, median, sum)
- **Cross Tab** → counting

13. Operations

13.1 Basic Operations

python

```
df.shape          # (rows, columns)
df.columns        # Column names
df.info()         # Data types, non-null counts
df.describe()     # Statistical summary
```

13.2 Broadcasting

python

```
df['A'] + 10      # Adds 10 to every value in column A
df['A'] * 2       # Multiplies every value by 2
```

13.3 Applying Functions

Custom function:

python

```
def square(x):
    return x * x

df['B'].apply(square)
```

Lambda function:

python

```
df['B'].apply(lambda x: x * x)
```

Creating a new column with applied function:

python

```
df['D'] = df['B'].apply(lambda x: x * x)
```

14. Feature Extraction (Project 1: Anime Dataset)

14.1 What is Feature Extraction?

- Extracting **new features** from existing columns.
- Example: Extracting **episode count** from a **title** column.

14.2 Extracting Episode Count

Function:

python

```
def extract_episodes(text):  
    check = False  
    data = ""  
    for i in text:  
        if i == ')':  
            check = False  
            break  
        if check:  
            data += i  
        if i == '(':  
            check = True  
    return data
```

Apply:

python

```
df['Episodes'] = df['Title'].apply(extract_episodes)
```

Clean:

python

```
df['Episodes'] = df['Episodes'].str.replace(' eps', '')
df['Episodes'] = df['Episodes'].astype(int)
```

14.3 Extracting Time Stamp

Function:

python

```
def extract_time(text):
    data = ""
    for i, char in enumerate(text):
        if char == ')':
            for j in range(i + 1, i + 19):
                data += text[j]
            break
    return data
```

Apply:

python

```
df['Total Time'] = df['Title'].apply(extract_time)
```

14.4 Extracting Months

Using ChatGPT (Prompt Engineering):

- Prompt: *"This is a column inside that we have data like [April 2009 - July 2010]. I want to extract total months from this data. My column name is Total Time."*
- ChatGPT provides a function using `dateutil` and `relativedelta`.

Apply:

python

```
df['Months'] = df['Total Time'].apply(calculate_total_months)
```

15. Data Extraction (Project 2: Countries Dataset)

15.1 Loading Data

python

```
import numpy as np
import pandas as pd

df = pd.read_csv('countries.csv')
```

15.2 Data Preprocessing

python

```
df.shape
df.info()
df.describe()
df.columns
```

- Check for **null values** in `info()` .
- Fill or remove null values before analysis.

15.3 Questions & Solutions

1. Which country has the highest population?

python

```
df[df['Population'] == df['Population'].max()]['Country']
```

2. What is the capital of the country with the highest population?

python

```
df[df['Population'] == df['Population'].max()]['Capital City']
```

3. Which country has the least population?

python

```
df[df['Population'] == df['Population'].min()]['Country']
```

4. What is the capital of the country with the least population?

python

```
df[df['Population'] == df['Population'].min()][ 'Capital City']
```

5. Top 5 countries with highest democratic score:

python

```
df.sort_values(by='Democratic Score', ascending=False, inplace=True)  
df['Country'].head()
```

6. How many total regions are there?

python

```
df['Region'].count()
```

7. How many countries lie in Eastern Europe region?

python

```
df[df['Region'] == 'Eastern Europe']['Country'].count()
```

8. Who is the political leader of the second-highest populated country?

python

```
df.nlargest(2, 'Population').iloc[1]['Political Leader']
```

9. How many countries have unknown political leaders?

python

```
df[df['Political Leader'].isna()][ 'Country'].count()
```

10. How many countries have "Republic" in their full name?

python

```
count = 0
```

```
def count_republic(text):  
    global count  
    if 'republic' in text.lower():  
        count += 1  
    return text  
  
df['Country Long'].apply(count_republic)  
print(count)
```

11. Which country in the African region has the highest population?

python

```
africa_df = df[df['Continent'] == 'Africa']  
africa_df[africa_df['Population'] == africa_df['Population'].max()][ 'Country']
```

16. Working with Real Data

16.1 Data Sources

- **CSV** (`pd.read_csv()`)
- **Excel** (`pd.read_excel()`)
- **JSON** (`pd.read_json()`)
- **SQL** (`pd.read_sql()`)
- **TSV, HTML, Parquet, Feather, Pickle, HDF5, ORC, Google Sheets**
- **Kaggle** — best source for datasets

16.2 Kaggle

- Website with thousands of datasets.
- Create an account and download datasets.
- Examples: Heart Attack, Cat Breed, Farm Production, Student Dropout, Spotify Songs, etc.

16.3 Loading a CSV

python

```
df = pd.read_csv(r'anime.csv')
df.head()
```

- Use **raw string** (`r'...'`) to avoid escape sequence issues.
- `df.head()` shows top 5 rows.

17. Key Takeaways

Concept	Description
Series	1D labeled array; can hold any data type
DataFrame	2D table; combination of multiple Series
Indexing	<code>.loc[]</code> (label), <code>.iloc[]</code> (position)
Conditional Selection	<code>df[df['col'] > value]</code>
Missing Data	<code>isna()</code> , <code>dropna()</code> , <code>fillna()</code>
Merging	<code>pd.merge()</code> — SQL-style joins
Concatenation	<code>pd.concat()</code> — stacking
Joining	<code>df.join()</code> — based on index
Group By	<code>df.groupby()</code> — aggregate by category
Aggregation	<code>mean()</code> , <code>median()</code> , <code>mode()</code> , <code>std()</code> , etc.
Pivot Table	<code>pd.pivot_table()</code> — reshape & aggregate
Cross Tab	<code>pd.crosstab()</code> — frequency counting
Apply	<code>df['col'].apply(func)</code> — apply custom function
Feature Extraction	Extract new features from existing columns
Data Extraction	Extract specific data using conditions

18. Next Steps

- Next video: **Data Visualization** (Matplotlib, Seaborn).
- Practice all exercises.
- Use **ChatGPT / AI tools** for prompt engineering when stuck.
- All notebooks available on **GitHub**.

Hashtag: #TillTheEnd



सो वी आर ऑन अ जर्नी टू कं...
TXT 419.08KB

make notes for me out of this transcript , explain everything properly deeply

Data Visualization — Complete Notes (from the Transcript)

Below are deep, structured notes covering **everything** in the transcript: Matplotlib, Seaborn, Plotly, Cufflinks, distribution plots, categorical plots, matrix plots, regression plots, and the IPL 2022 Capstone Project.

1. Introduction

- This is the **third video** in the Data Science series (after NumPy and Pandas).
- Topic: **Data Visualization**
- Libraries covered:
 - **Matplotlib** — core plotting library
 - **Seaborn** — statistical, stylish plots on top of Matplotlib
 - **Plotly** — interactive, clickable, zoomable charts
 - **Cufflinks** — connects Plotly with Pandas DataFrames
- Two parts:

1. Code Implementation

2. Project — IPL 2022 Analysis (Gujarat won that season)

Quote: *"Data is just numbers until you turn it into a picture."*

2. Origin Story of the Libraries

2.1 Matplotlib

- Created by **John Hunter** in **2003**.
- Goal: Bring **MATLAB**-like plotting to Python.
- **MATLAB** = high-level programming language & numerical computing environment used by engineers & scientists.
- Powerful but **complex** for beginners.

2.2 Seaborn

- Created by **Michael Waskom**, a neuroscientist.
- Built **on top of Matplotlib**.
- Goal: Simpler, more beautiful **statistical charts**.
- Like a stylish, easy upgrade over Matplotlib.

2.3 Cufflinks

- Created by **Santiago Palladino**.
- Built **on top of Seaborn**.
- Enables **interactive, clickable, zoomable charts** — even in 3D — with one line of code.
- Useful for **dashboards** and **non-technical audiences**.

3. Code vs Tools

Aspect	Code (Matplotlib, Seaborn, Plotly)	Tools (Power BI, Tableau, Google Data Studio)
Flexibility	Very flexible; customize every element	Limited to what the tool allows
Integration	Easy to integrate into ML pipelines & apps	Hard to integrate with ML pipelines
Code needed	Yes	No (drag & drop)
Use case	Data Science, ML, automation	Business dashboards, quick reporting

This video focuses on the Code part. Tools will be covered later.

4. What You Will Learn

- Matplotlib basics
- Subplots
- Object-Oriented way
- Types of plots
- Saving plots
- Working with images
- Optional designing
- **Seaborn:**
 - Distribution plots
 - Categorical plots
 - Matrix plots
 - Regression plots
- **Plotly & Cufflinks** (brief)
- **Capstone Project:** IPL 2022 Analysis

5. Matplotlib Basics

5.1 Imports

python

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

5.2 Creating Basic Arrays

python

```
x = np.linspace(0, 5, 11)  # 0 to 5, 11 elements
y = x ** 2                # Square of x
```

5.3 Basic Line Plot

python

```
plt.plot(x, y)
plt.title("Hello")
plt.xlabel("X Axis")
plt.ylabel("Y Axis")
plt.show()
```

- **X-axis** = horizontal
- **Y-axis** = vertical
- `plt.plot()` creates a line graph.

5.4 Labels & Titles

python

```
plt.title("Hello")
plt.xlabel("X Axis")
plt.ylabel("Y Axis")
```

6. Subplots

6.1 What is a Subplot?

- A **canvas** (figure) where you can place **multiple plots** in a grid.

python

```
plt.subplot(2, 2, 1)  # 2 rows, 2 columns, first plot
plt.plot(x, y)
```

```
plt.subplot(2, 2, 2)  # Second plot
plt.plot(y, x)
```

```
plt.subplot(2, 2, 3)  # Third plot
plt.plot(x, x**2)
```

```
plt.subplot(2, 2, 4)  # Fourth plot
plt.plot(x, y)
```

- subplot(rows, columns, position)
- Position numbering: **left-to-right, top-to-bottom**.

6.2 Limitations

- You **cannot move plots around** freely.
- The canvas behaves like a single image.
- To get **full control**, use the **Object-Oriented way**.

7. Object-Oriented Way

7.1 Creating a Figure (Canvas)

python

```
fig = plt.figure(figsize=(12, 8), dpi=100)
```

- `figsize` = (width, height) in inches
- `dpi` = dots per inch (resolution)
- Default: 6.4 × 4.8 inches, dpi=100

7.2 Adding Axes

python

```
ax1 = fig.add_axes([0.1, 0.1, 0.8, 0.8])  
ax1.plot(x, y)
```

- [left, bottom, width, height] — all values between 0 and 1.
- **left** = % from left
- **bottom** = % from bottom
- **width** = % width of canvas
- **height** = % height of canvas

7.3 Multiple Axes

python

```
ax1 = fig.add_axes([0.1, 0.1, 0.8, 0.8])  
ax2 = fig.add_axes([0.3, 0.4, 0.4, 0.3])
```

- You can place axes **anywhere** on the canvas.
- Full control over position and size.

8. Types of Plots in Matplotlib

8.1 Scatter Plot

python

```
plt.scatter(x, y)
```

- Shows **individual data points**.

- Used in **regression** and **correlation** analysis.

8.2 Histogram

python

```
plt.hist(data)
```

- Shows **distribution** of a single variable.
- Only needs **x** (no **y**).

8.3 Box Plot

python

```
plt.boxplot(x)
```

- Shows **5-number summary**: min, Q1, median, Q3, max.
- Used for **outlier detection**.

8.4 Saving Plots

python

```
fig.savefig('basic_plot.png')
```

- Saves the figure as an image.

9. Working with Images (Matplotlib)

9.1 Loading an Image

python

```
import matplotlib.image as mpimg  
img = mpimg.imread('image.jpeg')
```

- Image is converted into a **NumPy array** of pixel values.

9.2 Displaying an Image

```
python

plt.imshow(img)
plt.axis('off')
plt.show()
```

9.3 Cropping an Image

```
python

cropped = img[50:200, 50:200]
plt.imshow(cropped)
```

- Uses **NumPy slicing**.
- Used heavily in **Deep Learning (CNN)**.

10. Optional Designing

```
python

plt.plot(x, y, color='pink', linewidth=2, linestyle='--', marker='o')
```

- `color` — line color
- `linewidth` — thickness
- `linestyle` — solid, dashed, dotted
- `marker` — circle, triangle, square, etc.

Note: This is optional — not the main focus.

11. Seaborn — Distribution Plots

11.1 What is a Distribution Plot?

- Used for **quantitative (continuous)** variables.

- Examples: Age, Salary, Total Bill, Tips.

11.2 Types

- Histogram (histplot / distplot)
- Joint Plot
- Pair Plot
- Rug Plot

11.3 Loading Seaborn Data

python

```
import seaborn as sns
df = sns.load_dataset('tips')
```

- Seaborn has built-in datasets.
- `tips` dataset: total_bill, tip, sex, smoker, day, time, size.

11.4 Identifying Variable Types

Variable	Type
total_bill	Quantitative
tip	Quantitative
sex	Qualitative (Categorical)
smoker	Qualitative
day	Qualitative
time	Qualitative
size	Qualitative (only 6 unique values)

Rule: Just because a column has numbers doesn't mean it's quantitative. Check **unique values**.

11.5 Histogram / Dist Plot

python

```
sns.histplot(df['total_bill'], bins=20, kde=True)
```

- `bins` — number of bins
- `kde=True` — adds **Kernel Density Estimator** curve (bell curve)

11.6 Joint Plot

python

```
sns.jointplot(x='total_bill', y='tip', data=df, kind='scatter')
```

- Compares **two quantitative variables**.
- Shows **scatter plot + distribution** on top and side.
- `kind` options: `scatter`, `kde`, `hex`, `reg`, `resid`

11.7 Pair Plot

python

```
sns.pairplot(df, hue='sex')
```

- Compares **all numerical columns** with each other.
- `hue` — differentiates by category (e.g., male/female).
- Great for **initial data exploration**.

11.8 Rug Plot

python

```
sns.rugplot(df['total_bill'])
```

- Shows **individual data points** as small sticks.
- Rarely used alone.

12. Seaborn — Categorical Plots

12.1 What is a Categorical Plot?

- Used for **qualitative (categorical)** variables.
- Examples: Gender, Day, Smoker, Time.

12.2 Types

- Count Plot
- Bar Plot
- Box Plot
- Violin Plot
- Strip Plot
- Swarm Plot

12.3 Count Plot

python

```
sns.countplot(x='sex', data=df)
sns.countplot(x='sex', hue='smoker', data=df)
```

- Counts occurrences of each category.
- `hue` — splits by another category.

12.4 Bar Plot

python

```
sns.barplot(x='sex', y='total_bill', data=df, estimator=np.sum)
```

- `x` = categorical, `y` = quantitative.
- Default estimator = **mean**.
- Can change to `np.sum`, `np.median`, `np.min`, `np.max`, `np.std`.

12.5 Box Plot

python

```
sns.boxplot(x='tip', y='day', data=df, palette='rainbow')
```

- Shows **distribution** of quantitative data across categories.
- Shows **outliers** (points outside whiskers).

12.6 Violin Plot

python

```
sns.violinplot(x='day', y='total_bill', data=df)
```

- Combines **box plot + KDE**.
- Shows **spread** and **distribution shape**.

12.7 Strip Plot

python

```
sns.stripplot(x='day', y='total_bill', data=df)
```

- Draws a **scatter plot** where one variable is categorical.

12.8 Swarm Plot

python

```
sns.swarmplot(x='day', y='total_bill', data=df)
```

- Like strip plot but **points don't overlap**.
- Better representation of distribution.

12.9 Combining Violin + Swarm

python

```
sns.violinplot(x='day', y='total_bill', data=df)
sns.swarmplot(x='day', y='total_bill', data=df, color='black')
```

- Both on the same canvas.

13. Seaborn — Matrix Plots

13.1 What is a Matrix Plot?

- Uses **matrices** (rows × columns) instead of x/y axes.

13.2 Types

- Heat Map
- Cluster Map
- Pivot Table Heat Map

13.3 Correlation

- **Correlation** measures relationship between two variables.
- Range: **-1 to +1**
 - **+1** = perfect positive correlation
 - **-1** = perfect negative correlation
 - **0** = no correlation

python

```
tips_corr = df[['total_bill', 'tip', 'size']].corr()
```

13.4 Heat Map

python

```
sns.heatmap(tips_corr, annot=True)
```

- `annot=True` — shows values inside cells.
- Darker colors = higher correlation.

13.5 Cluster Map

python

```
sns.clustermap(tips_corr)
```

- Like heatmap but with **hierarchical clustering** tree.
- Uses **Hierarchical Algorithm** (covered in ML).

13.6 Pivot Table Heat Map

python

```
flights = sns.load_dataset('flights')  
pivot_flight = flights.pivot_table(values='passengers', index='month', columns='year')  
sns.heatmap(pivot_flight)
```

- Pivot table → heatmap.
- Shows **year vs month vs passengers**.

14. Seaborn — Regression Plots

14.1 What is a Regression Plot?

- Used in **Machine Learning**.
- Shows **scatter plot + best fit line**.
- The best fit line is found using **Linear Regression**.

14.2 LM Plot

python

```
sns.lmplot(x='total_bill', y='tip', data=tips, hue='sex', palette='coolwarm')
```

- **x** = independent variable (predictor)
- **y** = dependent variable (target)
- `hue` — separate lines for categories
- `markers` — change marker shapes
- `scatter_kws={'s': 100}` — change marker size

14.3 Other Parameters

- `palette` — color palette
- `markers` — `['o', 'v']` (circle, inverted triangle)
- `scatter_kws` — size, transparency, etc.

15. Plotly & Cufflinks (Brief)

15.1 Installation

```
bash
```

```
pip install plotly  
pip install cufflinks
```

15.2 Imports

```
python
```

```
import plotly.offline as pyo  
from plotly.offline import iplot  
import cufflinks as cf  
cf.go_offline()
```

15.3 Interactive Line Plot

```
python
```

```
df['total_bill'].iplot(kind='line')
```

15.4 Interactive Bar Plot

python

```
df.groupby('day')['tip'].mean().plot(kind='bar')
```

15.5 Interactive Scatter Plot

python

```
df.plot(kind='scatter', x='total_bill', y='tip', mode='markers')
```

15.6 Interactive Box Plot

python

```
df[['total_bill', 'day']].plot(kind='box', x='total_bill', y='day')
```

Advantages:

- Hover to see exact values
- Zoom in/out
- Export to Plotly
- Better for presentations

16. Capstone Project: IPL 2022 Analysis

16.1 Dataset

- IPL 2022 match-level data (74 matches, 20 columns).
- Columns: match_id, date, venue, team1, team2, stage, toss_winner, toss_decision, first_innings_score, second_innings_score, match_winner, won_by, margin, player_of_the_match, top_scorer, highest_score, best_bowling, best_bowling_figure, etc.

16.2 Loading Libraries & Data

```
python
```

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')

df = pd.read_csv('IPL.csv')
df.head()
```

16.3 Basic Info

```
python
```

```
df.info()
df.shape
df.isnull().sum()
```

16.4 Questions & Solutions

Q1: Which team won the most matches?

```
python
```

```
match_wins = df['match_winner'].value_counts()
sns.barplot(y=match_wins.index, x=match_wins.values, palette='viridis')
plt.title('Most Match Wins by Team')
```

→ **Gujarat Titans** won 12 matches.

Q2: Toss Decision Trends

```
python
```

```
sns.countplot(x='toss_decision', data=df, palette='rainbow')
plt.title('Toss Decision Trends')
```

→ Most teams chose to **field** first (60+ times) rather than bat.

Q3: Toss Winner vs Match Winner

python

```
count = df[df['toss_winner'] == df['match_winner']]['match_id'].count()
percentage = (count * 100) / df.shape[0]
print(f"Toss winner also won the match: {percentage:.2f}%")
```

→ Only **48.65%** chance that winning the toss means winning the match.

Q4: How do teams win (Runs vs Wickets)?

python

```
sns.countplot(x='won_by', data=df)
plt.title('Won By')
```

→ 37 matches won by **wickets**, 37 by **runs** — perfectly balanced.

Q5: Top 10 Player of the Match Awards

python

```
count = df['player_of_the_match'].value_counts().head(10)
sns.barplot(x=count.values, y=count.index, palette='mako')
plt.title('Top 10 Players with Man of the Match')
```

→ **Kuldeep Yadav** won 4 awards.

Q6: Top 2 Scorers

python

```
high = df.groupby('top_scorer')['highest_score'].sum().sort_values(ascending=False).head(2)
high.plot(kind='barh')
```

→ **Jos Buttler** (651 runs), **Quinton de Kock** (377 runs).

Q7: 10 Best Bowling Figures

python

```
df['highest_wickets'] = df['best_bowling_figure'].apply(lambda x: x.split('--')[0])
df['highest_wickets'] = df['highest_wickets'].astype(int)
```

```
top_bowlers = df.groupby('best_bowling')['highest_wickets'].sum().sort_values(ascending=False).head(10)
top_bowlers.plot(kind='barh')
```

→ **Yuzvendra Chahal** (17 wickets), **Kuldeep Yadav** (11), **Rashid Khan** (11).

Q8: Most Matches Played by Venue

python

```
venue_count = df['venue'].value_counts()
sns.barplot(y=venue_count.index, x=venue_count.values, palette='rainbow')
plt.title('Most Matches Played by Venue')
```

→ **Wankhede Stadium** (20 matches), **DY Patil Academy**, **Brabourne Stadium**.

Q9: Highest Margin by Runs

python

```
df[df['won_by'] == 'Runs'].sort_values(by='margin', ascending=False).head(1)[['match_winner', 'margin']]
```

→ **Chennai Super Kings** won by **91 runs**.

Q10: Highest Individual Score

python

```
df[df['highest_score'] == df['highest_score'].max()][['top_scorer', 'highest_score']]
```

→ **Quinton de Kock** scored **140 runs**.

Q11: Best Bowling Figures

python

```
df[df['highest_wickets'] == df['highest_wickets'].max()][['best_bowling', 'best_bowling_figure']]
```

→ **Jasprit Bumrah** (5/10), **Yuzvendra Chahal** (5/40), etc.

17. Key Takeaways

Concept	Description
Matplotlib	Core plotting library; basic plots, subplots, OO way
Seaborn	Statistical plots; distribution, categorical, matrix, regression
Plotly	Interactive plots
Cufflinks	Connects Plotly with Pandas
Distribution Plots	Histogram, Joint Plot, Pair Plot, Rug Plot
Categorical Plots	Count, Bar, Box, Violin, Strip, Swarm
Matrix Plots	Heatmap, Clustermap, Pivot Table Heatmap
Regression Plots	LM Plot — scatter + best fit line
Feature Extraction	Extracting new features from existing columns
Data Analysis	Asking questions and answering with code

18. Next Steps

- Next video: **Statistics** (highly recommended before ML).
- Then: **Machine Learning** series.
- Practice on **Kaggle** datasets.
- All notebooks available on **GitHub**.

Hashtag: #IPL

give me interview questions with answers topic wise of all the things covered in this chat

Data Science Interview Questions & Answers — Topic Wise

Based on everything covered in this chat (NumPy, Pandas, Data Visualization), here are comprehensive interview questions with answers.

PART 1: NUMPY

Topic 1: NumPy Basics & Origin

Q1: What is NumPy? Why is it faster than Python lists?

Answer:

NumPy (Numerical Python) is a Python library for numerical computing. It provides:

- Multidimensional arrays (ndarray)
- Mathematical functions
- Broadcasting capabilities

Why faster:

- NumPy is written in **C** (compiled language) — Python is **interpreted**
- NumPy uses **contiguous memory blocks** — Python lists use scattered memory
- NumPy supports **vectorized operations** — no need for explicit loops
- NumPy arrays are up to **50x faster** than Python lists

Q2: What is the origin story of NumPy?

Answer:

- **1995:** Jim Hugunin created **Numeric** library — provided array support but poor performance
- **Early 2000s:** Space Telescope Science Institute developed **Numarray** — better but incompatible with Numeric
- **2005:** **Travis Oliphant** combined Numeric + Numarray → created **NumPy**
- Finally, Python had proper arrays.

Q3: What is the difference between a Python list and a NumPy array?**Answer:**

Feature	Python List	NumPy Array
Data Types	Heterogeneous (mixed)	Homogeneous (same type)
Performance	Slow	Fast (50x)
Memory	Low efficiency	High efficiency
Vector Operations	Manual loops	Broadcasting
Size	Dynamic	Fixed

Topic 2: Array Creation**Q4: How do you create a NumPy array? Name all methods.****Answer:**

python

```
import numpy as np
```

```
# From List
```

```
np.array([1, 2, 3, 4])
```

```
# From tuple
```

```
np.array((1, 2, 3))
```

```
# Zeros
```

```
np.zeros((3, 4))
```

```
# Ones
```

```
np.ones((3, 4))
```

```
# Range
```

```
np.arange(1, 11, 2)
```

```
# Linear spacing
```

```
np.linspace(0, 1, 100)
```

```
# Identity matrix
np.eye(3)

# Random
np.random.rand(3, 4)
np.random.randn(3, 4)
np.random.randint(1, 100, 10)
```

Q5: What is the difference between `np.arange()` and `np.linspace()` ?

Answer:

Feature	<code>np.arange()</code>	<code>np.linspace()</code>
Purpose	Range with step	Evenly spaced numbers
Parameters	start, stop, step	start, stop, num
Stop value	Exclusive	Inclusive
Example	<code>np.arange(0, 10, 2) → [0,2,4,6,8]</code>	<code>np.linspace(0, 10, 5) → [0, 2.5, 5, 7.5, 10]</code>

Q6: What is the difference between `np.random.rand()` and `np.random.randn()` ?

Answer:

- `np.random.rand()` → Uniform distribution between **0 and 1** (normalization)
- `np.random.randn()` → Standard normal distribution (mean=0, variance=1), values typically between **-3 and +3** (standardization)

Q7: How do you create a 2D matrix in NumPy?

Answer:

```
python

# From nested list
arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

# Using reshape
arr = np.arange(1, 10).reshape(3, 3)
```

```
# Using zeros/ones
arr = np.zeros((3, 3))
arr = np.ones((3, 3))
```

Topic 3: Array Attributes & Methods

Q8: What are the key attributes of a NumPy array?

Answer:

python

```
arr = np.array([[1, 2, 3], [4, 5, 6]])

arr.shape    # (2, 3) — rows × columns
arr.size     # 6 — total elements
arr.ndim     # 2 — number of dimensions
arr.dtype    # int64 — data type
arr.itemsize # 8 — bytes per element
```

Q9: What is the difference between attributes and methods in NumPy?

Answer:

- **Attributes:** No brackets () — e.g., `arr.shape`, `arr.size`, `arr.dtype`
- **Methods:** Use brackets () — e.g., `arr.min()`, `arr.max()`, `arr.sum()`, `arr.mean()`

Q10: How do you find the index of max/min element in a NumPy array?

Answer:

python

```
arr = np.array([10, 5, 20, 15])

arr.argmax() # 2 (index of 20)
arr.argmin() # 1 (index of 5)
```

Q11: What is the difference between `axis=0` and `axis=1` in NumPy?

Answer:

- `axis=0` → **Column-wise** operation (vertical)
- `axis=1` → **Row-wise** operation (horizontal)

python

```
arr = np.array([[1, 2, 3], [4, 5, 6]])

np.sum(arr, axis=0) # [5, 7, 9] - column sums
np.sum(arr, axis=1) # [6, 15] - row sums
```

Topic 4: Reshaping & Resizing

Q12: What is the difference between `reshape()` and `resize()` ?

Answer:

- `reshape()` → Returns a **new array** with new shape (original unchanged)
- `resize()` → Changes the array **in-place** (original modified)

python

```
arr = np.arange(1, 13)

reshaped = arr.reshape(3, 4) # New array
arr.resize(3, 4)             # In-place
```

Q13: What happens if you try to reshape an array into an incompatible shape?

Answer:

You get a **ValueError**. The total number of elements must match.

python

```
arr = np.arange(1, 31) # 30 elements
arr.reshape(5, 5)      # ERROR: 25 ≠ 30
```

Topic 5: Indexing & Slicing

Q14: How do you index and slice a 1D NumPy array?

Answer:

python

```

arr = np.arange(11, 21)

arr[0]      # 11
arr[5]      # 16
arr[1:5]    # [12, 13, 14, 15]
arr[:5]     # [11, 12, 13, 14, 15]
arr[3:]     # [14, 15, 16, 17, 18, 19, 20]
arr[1:9:2]  # [12, 14, 16, 18]

```

Q15: How do you index and slice a 2D NumPy array?**Answer:**

python

```

arr = np.arange(1, 31).reshape(6, 5)

arr[0]      # First row
arr[0, 0]   # Element at row 0, col 0
arr[0, 4]   # Element at row 0, col 4
arr[0:2, 1:3] # Rows 0-1, Cols 1-2
arr[:, 2]    # Entire column 2
arr[3:, 3:]  # Rows 3-end, Cols 3-end

```

Q16: What is Boolean Indexing in NumPy?**Answer:**

Filtering arrays using boolean conditions.

python

```

arr = np.arange(11, 21)
bool_idx = arr % 2 == 0
arr[bool_idx] # [12, 14, 16, 18, 20]

# Direct
arr[arr > 15] # [16, 17, 18, 19, 20]

```

Rule: Boolean array must have the same length as the original array.

Topic 6: Broadcasting

Q17: What is Broadcasting in NumPy?

Answer:

Broadcasting allows operations between arrays of **different shapes** by "stretching" the smaller array.

python

```
arr = np.array([10, 20, 30, 40])
arr + 10 # [20, 30, 40, 50]

# Works for matrices too
arr2 = np.arange(1, 26).reshape(5, 5)
arr2 + 10 # Adds 10 to every element
```

Rules:

1. If dimensions differ, prepend 1s to the smaller shape.
2. Arrays are compatible if dimensions are equal or one of them is 1.
3. The smaller array is broadcast across the larger one.

Topic 7: Deep vs Shallow Copy

Q18: What is the difference between Deep Copy and Shallow Copy in NumPy?

Answer:

Feature	Shallow Copy (<code>b = a</code>)	Deep Copy (<code>b = a.copy()</code>)
Memory	Same memory location	Separate memory
Changes	Affect both	Affect only the copy
Method	<code>b = a</code>	<code>b = a.copy()</code>

Note: In NumPy, slicing creates a **view** (shallow copy) — changes affect the original.

python

```
a = np.array([1, 2, 3, 4, 5])
b = a          # Shallow
c = a.copy()   # Deep

b[0] = 99
print(a[0])    # 99 (affected)
print(c[0])    # 1 (not affected)
```

Topic 8: Matrix Operations

Q19: How do you perform matrix multiplication in NumPy?

Answer:

python

```
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])

a @ b          # Matrix multiplication
np.dot(a, b)   # Same
np.matmul(a, b) # Same
a * b          # Element-wise multiplication (NOT matrix multiplication)
```

Rule: Columns of first matrix must equal rows of second matrix.

Q20: What is Transpose in NumPy?

Answer:

Transpose swaps rows and columns.

python

```
a = np.array([[1, 2], [3, 4]])
a.T # [[1, 3], [2, 4]]
```

Topic 9: Stacking & Splitting

Q21: How do you stack arrays in NumPy?

Answer:

```
python

a = np.array([1, 2, 3, 4])
b = np.array([5, 6, 7, 8])

np.vstack((a, b))      # Vertical stack
np.hstack((a, b))      # Horizontal stack
np.column_stack((a, b)) # Columns
```

Note: Only works with **vectors (1D arrays)**, not matrices.

Q22: How do you split arrays in NumPy?**Answer:**

```
python

c = np.arange(1, 17).reshape(4, 4)

np.hsplit(c, 2)  # Horizontal split
np.vsplit(c, 2)  # Vertical split
np.split(c, 4)   # Split into 4 parts
```

Rule: Must split into **equal divisions**.

 PART 2: PANDAS**Topic 1: Pandas Basics & Origin****Q23: What is Pandas? What is the origin of its name?****Answer:**

Pandas is a Python library for data manipulation and analysis.

Origin:

- **2008:** Wes McKinney started developing Pandas at a financial company.

- Name comes from **"Panel Data"** — an econometrics term for multidimensional datasets.
- Not named after the animal.

Q24: What is the difference between a Series and a DataFrame?

Answer:

Feature	Series	DataFrame
Dimensions	1D	2D
Structure	Labeled array	Table (rows × columns)
Data Types	Any	Multiple columns with different types
Creation	<code>pd.Series()</code>	<code>pd.DataFrame()</code>

python

```
# Series
```

```
s = pd.Series([10, 20, 30], index=['A', 'B', 'C'])
```

```
# DataFrame
```

```
df = pd.DataFrame({'Name': ['John', 'Anna'], 'Age': [28, 34]})
```

Topic 2: DataFrame Creation & Selection

Q25: How do you create a DataFrame?

Answer:

python

```
# From dictionary
```

```
df = pd.DataFrame({  
    'Name': ['John', 'Anna', 'Peter'],  
    'Age': [28, 34, 29],  
    'City': ['NY', 'Paris', 'Berlin']  
})
```

```
# From List
```

```
df2 = pd.DataFrame(  
    [['John', 28], ['Anna', 34]],  
    columns=['Name', 'Age']  
)
```

Q26: How do you select columns in Pandas?

Answer:

python

```
df['Name']          # Single column (Series)  
df[['Name', 'Age']] # Multiple columns (DataFrame)
```

Q27: How do you select rows in Pandas?

Answer:

python

```
df.loc[0]          # By label  
df.iloc[0]         # By position  
df.loc[[0, 1]]     # Multiple rows  
df.loc[0:2]        # Slice
```

Q28: How do you select subsets of rows and columns?

Answer:

python

```
df.loc[[0, 1], ['Name', 'City']]
```

- First list → rows
- Second list → columns

Topic 3: Conditional Selection

Q29: How do you filter DataFrames based on conditions?

Answer:

python

```
# Single condition
df[df['Age'] > 30]

# Multiple conditions (AND)
df[(df['Age'] > 30) & (df['City'] == 'Paris')]

# Multiple conditions (OR)
df[(df['Age'] > 30) | (df['City'] == 'Paris')]
```

Rule: Wrap each condition in parentheses.

Topic 4: Missing Data

Q30: How do you find, remove, and fill missing data in Pandas?

Answer:

python

```
# Find
df.isna()           # Boolean DataFrame
df.isna().sum()     # Count per column
df.isna().any()     # True if any NaN

# Remove
df.dropna()         # Drop rows with ANY NaN
df.dropna(thresh=3) # Keep rows with ≥3 non-NaN
df.dropna(axis=1)   # Drop columns with NaN

# Fill
df.fillna(0)        # Fill with 0
df.fillna({'A': 0, 'B': 100}) # Different values per column
df.fillna(df.mean()) # Fill with mean
df.fillna(0, inplace=True) # Apply permanently
```

Topic 5: Merging, Joining & Concatenation

Q31: What is the difference between merge , join , and concat ?

Answer:

Operation	Based On	Use Case
merge()	Columns	SQL-style joins
join()	Index	Joining on index
concat()	Axis	Stacking (rows or columns)

python

Merge

```
pd.merge(df1, df2, on='EmployeeID', how='inner')
```

Join

```
df1.join(df2, how='outer')
```

Concat

```
pd.concat([df1, df2], axis=0) # Vertical
```

```
pd.concat([df1, df2], axis=1) # Horizontal
```

Q32: What are the types of joins in Pandas merge?**Answer:**

Type	Description
inner (default)	Only matching rows
outer	All rows from both; NaN where no match
left	All rows from left; NaN where no match on right
right	All rows from right; NaN where no match on left

Topic 6: Group By & Aggregation**Q33: What is `groupby()` in Pandas?**

Answer:

Groups data by a categorical column and performs operations on each group.

python

```
df.groupby('Category')['Sales'].sum()
df.groupby(['Category', 'Store'])['Sales'].sum()
```

Note: Returns a GroupBy object (not a DataFrame).

Q34: What are common aggregation functions in Pandas?**Answer:**

python

```
df['Sales'].mean()
df['Sales'].median()
df['Sales'].min()
df['Sales'].max()
df['Sales'].count()
df['Sales'].std()
df['Sales'].var()

# Multiple at once
df['Sales'].agg(['mean', 'median', 'min', 'max', 'count', 'std'])
```

Q35: What is the difference between qualitative and quantitative variables?**Answer:**

Type	Description	Examples
Qualitative (Categorical)	Categories	Gender, Day, Smoker
Quantitative (Continuous)	Numerical values	Age, Salary, Sales

Rule: Just because a column has numbers doesn't mean it's quantitative. Check unique values.

Topic 7: Pivot Tables & Cross Tabs

Q36: What is a Pivot Table?

Answer:

Reshapes data — creates a new DataFrame with index, columns, and aggregated values.

python

```
pd.pivot_table(df, values='Sales', index='Region', columns='Product', aggfunc='mean')
```

- Default aggregation = **mean**
- Can change to `median`, `sum`, `count`, etc.
- **Values** must be numeric.

Q37: What is the difference between Pivot Table and Cross Tab?

Answer:

Feature	Pivot Table	Cross Tab
Purpose	Aggregation (mean, sum)	Counting
Function	<code>pd.pivot_table()</code>	<code>pd.crosstab()</code>
Values	Must be numeric	Any

python

```
pd.crosstab(df['Region'], df['Product'])
```

Topic 8: Apply & Feature Extraction

Q38: What is the `apply()` function in Pandas?

Answer:

Applies a function to each element of a column.

python

```
def square(x):  
    return x * x
```

```
df['B'].apply(square)

# Lambda
df['B'].apply(lambda x: x * x)
```

Q39: What is Feature Extraction?

Answer:

Extracting new features from existing columns.

Example: Extracting episode count from a title column.

```
python

def extract_episodes(text):
    # logic to extract episodes
    return episodes

df['Episodes'] = df['Title'].apply(extract_episodes)
```

Q40: How do you extract data using conditions in Pandas?

Answer:

```
python

# Highest population country
df[df['Population'] == df['Population'].max()]['Country']

# Top 2 countries by democratic score
df.sort_values(by='Democratic Score', ascending=False).head(2)

# Countries with unknown political leaders
df[df['Political Leader'].isna()]['Country'].count()
```

PART 3: DATA VISUALIZATION

Topic 1: Matplotlib Basics

Q41: What is Matplotlib? Who created it?

Answer:

Matplotlib is a Python plotting library created by **John Hunter in 2003**. It was designed to bring MATLAB-like plotting to Python.

Key features:

- Line plots, scatter plots, bar charts, histograms
- Subplots
- Object-oriented interface
- Customization of every element

Q42: What is the difference between `plt.plot()` and `plt.scatter()` ?**Answer:**

Feature	<code>plt.plot()</code>	<code>plt.scatter()</code>
Purpose	Line graph	Individual points
Connection	Points connected by lines	No lines
Use case	Trends over time	Correlation, regression

Q43: What are subplots in Matplotlib?**Answer:**

Subplots allow multiple plots in a single canvas (figure).

```
python
```

```
plt.subplot(2, 2, 1) # 2 rows, 2 columns, first plot
plt.plot(x, y)
```

```
plt.subplot(2, 2, 2) # Second plot
plt.plot(y, x)
```

Q44: What is the Object-Oriented way in Matplotlib?**Answer:**

Creating a figure (canvas) and adding axes manually.

```
python
```

```
fig = plt.figure(figsize=(12, 8))
ax1 = fig.add_axes([0.1, 0.1, 0.8, 0.8])
ax1.plot(x, y)
```

- [left, bottom, width, height] — all between 0 and 1.
- Gives **full control** over plot placement.

Q45: How do you save a plot in Matplotlib?

Answer:

python

```
fig.savefig('plot.png')
```

Topic 2: Seaborn — Distribution Plots

Q46: What are Distribution Plots? When do you use them?

Answer:

Distribution plots visualize the distribution of **quantitative (continuous)** variables.

Types:

- Histogram (histplot / distplot)
- Joint Plot
- Pair Plot
- Rug Plot

python

```
sns.histplot(df['total_bill'], bins=20, kde=True)
```

Q47: What is the difference between Histogram and KDE?

Answer:

Feature	Histogram	KDE
Representation	Bars (bins)	Smooth curve
Purpose	Count/frequency	Probability density
Bins	Required	Not required

KDE = Kernel Density Estimator — shows the **bell curve** of normal distribution.

Q48: What is a Joint Plot?

Answer:

Compares **two quantitative variables** — shows scatter plot + distribution on top and side.

python

```
sns.jointplot(x='total_bill', y='tip', data=df, kind='scatter')
```

kind options: scatter , kde , hex , reg , resid

Q49: What is a Pair Plot?

Answer:

Compares **all numerical columns** with each other.

python

```
sns.pairplot(df, hue='sex')
```

- hue — differentiates by category
- Great for **initial data exploration**

Topic 3: Seaborn — Categorical Plots

Q50: What are Categorical Plots? Name all types.

Answer:

Categorical plots visualize **qualitative (categorical)** variables.

Types:

- Count Plot
- Bar Plot
- Box Plot
- Violin Plot
- Strip Plot
- Swarm Plot

Q51: What is the difference between Count Plot and Bar Plot?**Answer:**

Feature	Count Plot	Bar Plot
Purpose	Count occurrences	Aggregate (mean, sum)
x-axis	Categorical	Categorical
y-axis	Count	Quantitative
Function	<code>sns.countplot()</code>	<code>sns.barplot()</code>

Q52: What is a Box Plot? What is the 5-number summary?**Answer:**

Box plot shows distribution using **5-number summary**:

1. **Minimum**
2. **Q1** (25th percentile)
3. **Median** (Q2, 50th percentile)
4. **Q3** (75th percentile)
5. **Maximum**

Also shows **outliers** (points outside whiskers).

python

```
sns.boxplot(x='day', y='total_bill', data=df)
```

Q53: What is the difference between Violin Plot and Box Plot?**Answer:**

Feature	Box Plot	Violin Plot
Shape	Box + whiskers	Violin (KDE)
Distribution	Not shown	Shown (density)
Outliers	Shown	Not explicitly
Data spread	Quartiles	Full distribution

Q54: What is the difference between Strip Plot and Swarm Plot?**Answer:**

Feature	Strip Plot	Swarm Plot
Points	Overlap	Don't overlap
Distribution	Less clear	Clearer
Function	<code>sns.stripplot()</code>	<code>sns.swarmplot()</code>

Topic 4: Seaborn — Matrix Plots**Q55: What are Matrix Plots? Name all types.****Answer:**

Matrix plots use matrices instead of x/y axes.

Types:

- Heat Map
- Cluster Map
- Pivot Table Heat Map

Q56: What is Correlation? What is its range?

Answer:

Correlation measures the relationship between two variables.

Range: -1 to +1

- **+1** = perfect positive correlation
- **-1** = perfect negative correlation
- **0** = no correlation

python

```
tips_corr = df[['total_bill', 'tip', 'size']].corr()  
sns.heatmap(tips_corr, annot=True)
```

Q57: What is the difference between Heatmap and Clustermap?**Answer:**

Feature	Heatmap	Clustermap
Clustering	No	Yes (hierarchical)
Tree	No	Yes (dendrogram)
Purpose	Show correlation	Show clusters

Topic 5: Seaborn — Regression Plots**Q58: What is a Regression Plot?****Answer:**

Shows scatter plot + best fit line. Used in **Machine Learning** (Linear Regression).

python

```
sns.lmplot(x='total_bill', y='tip', data=tips, hue='sex')
```

- **x** = independent variable (predictor)
- **y** = dependent variable (target)

- Best fit line found using **Linear Regression**

Topic 6: Plotly & Cufflinks

Q59: What is Plotly? How is it different from Matplotlib?

Answer:

Feature	Matplotlib	Plotly
Interactivity	Static	Interactive
Zoom	No	Yes
Hover	No	Yes
3D	Limited	Yes
Dashboard	No	Yes

python

```
import plotly.offline as pyo
from plotly.offline import iplot
import cufflinks as cf
cf.go_offline()

df['total_bill'].iplot(kind='line')
```

Q60: What is Cufflinks?

Answer:

Cufflinks connects **Plotly** with **Pandas DataFrames**. It allows you to create interactive plots directly from DataFrames.

python

```
import cufflinks as cf
cf.go_offline()

df.iplot(kind='scatter', x='total_bill', y='tip')
```



PART 4: IPL 2022 CAPSTONE PROJECT

Q61: How do you analyze which team won the most matches?

Answer:

python

```
match_wins = df['match_winner'].value_counts()
sns.barplot(y=match_wins.index, x=match_wins.values)
```

→ **Gujarat Titans** won 12 matches.

Q62: How do you find the toss decision trend?

Answer:

python

```
sns.countplot(x='toss_decision', data=df)
```

→ Most teams chose to **field** first (60+ times).

Q63: What percentage of toss winners also win the match?

Answer:

python

```
count = df[df['toss_winner'] == df['match_winner']]['match_id'].count()
percentage = (count * 100) / df.shape[0]
```

→ **48.65%** — winning the toss doesn't guarantee winning the match.

Q64: How do you find the top 10 Player of the Match award winners?

Answer:

python

```
count = df['player_of_the_match'].value_counts().head(10)
sns.barplot(x=count.values, y=count.index)
```

→ **Kuldeep Yadav** won 4 awards.

Q65: How do you find the top 2 scorers?

Answer:

python

```
high = df.groupby('top_scorer')['highest_score'].sum().sort_values(ascending=False).head(2)
high.plot(kind='barh')
```

→ **Jos Buttler** (651 runs), **Quinton de Kock** (377 runs).

Q66: How do you find the 10 best bowling figures?

Answer:

python

```
df['highest_wickets'] = df['best_bowling_figure'].apply(lambda x: x.split('--')[0])
df['highest_wickets'] = df['highest_wickets'].astype(int)

top_bowlers = df.groupby('best_bowling')['highest_wickets'].sum().sort_values(ascending=False).head(10)
top_bowlers.plot(kind='barh')
```

→ **Yuzvendra Chahal** (17 wickets).

Q67: How do you find the highest margin by runs?

Answer:

python

```
df[df['won_by'] == 'Runs'].sort_values(by='margin', ascending=False).head(1)[['match_winner', 'margin']]
```

→ **Chennai Super Kings** won by **91** runs.

Q68: How do you find the highest individual score?

Answer:

python

```
df[df['highest_score'] == df['highest_score'].max()][['top_scorer', 'highest_score']]
```

→ **Quinton de Kock** scored **140** runs.



PART 5: GENERAL DATA SCIENCE CONCEPTS

Q69: What is Data Preprocessing?

Answer:

Data preprocessing is cleaning and transforming raw data into a usable format for analysis or ML.

Steps:

1. Handle missing values (fill or remove)
2. Remove duplicates
3. Handle outliers
4. Encode categorical variables
5. Scale/normalize numerical variables
6. Feature engineering

Q70: What is Feature Extraction vs Feature Engineering?

Answer:**Feature Extraction**

Extracting new features from existing

Example: Extract episode count from title

Automated

Feature Engineering

Creating new features from domain knowledge

Example: Create "age group" from age

Manual/creative

Q71: What is the difference between Normalization and Standardization?**Answer:**

Normalization	Standardization
Scales to [0, 1]	Scales to mean=0, std=1
Min-Max scaling	Z-score scaling
Sensitive to outliers	Less sensitive
$(x - \min) / (\max - \min)$	$(x - \text{mean}) / \text{std}$

Q72: What is the difference between Qualitative and Quantitative data?**Answer:**

Qualitative	Quantitative
Categorical	Numerical
Gender, Color, Day	Age, Salary, Height
Count Plot, Bar Plot	Histogram, Scatter Plot

Q73: What is the difference between `loc` and `iloc` ?**Answer:**

<code>loc</code>	<code>iloc</code>
Label-based	Integer position-based
<code>df.loc[0]</code>	<code>df.iloc[0]</code>
Includes end label	Excludes end index
Can use boolean arrays	Can use boolean arrays

Q74: What is the difference between `apply()` and `applymap()` ?**Answer:**

<code>apply()</code>	<code>applymap()</code>
Works on rows/columns	Works on every element
<code>df['col'].apply(func)</code>	<code>df.applymap(func)</code>
Returns Series/DataFrame	Returns DataFrame

Q75: What is the difference between `map()` and `applymap()` ?

Answer:

<code>map()</code>	<code>applymap()</code>
Works on Series	Works on DataFrame
<code>s.map(func)</code>	<code>df.applymap(func)</code>
Element-wise	Element-wise

BONUS: Scenario-Based Questions

Q76: You have a dataset with 1 million rows and missing values. How do you handle it?

Answer:

1. Check percentage of missing values per column.
2. If <5% — drop rows.
3. If 5-30% — fill with mean/median/mode.
4. If >30% — consider dropping the column.
5. Use advanced imputation (KNN, regression) if needed.
6. Always document your approach.

Q77: How do you detect outliers in a dataset?

Answer:

1. **Box Plot** — visual detection
2. **IQR Method** — values outside $Q1 - 1.5IQR$ and $Q3 + 1.5IQR$

- 3. **Z-Score** — values with $|z| > 3$
- 4. **Scatter Plot** — visual detection

Q78: You need to merge two DataFrames with different column names. How?

Answer:

```
python

pd.merge(df1, df2, left_on='ID1', right_on='ID2', how='inner')
```

Q79: How do you speed up a slow Pandas operation?

Answer:

- 1. Use vectorized operations (avoid loops)
- 2. Use `apply()` with `raw=True` for NumPy functions
- 3. Use `eval()` and `query()` for large DataFrames
- 4. Convert to NumPy arrays for heavy computation
- 5. Use `category` dtype for categorical columns
- 6. Use `chunksize` for reading large files

Q80: What is the difference between `pd.concat()` and `pd.merge()` ?

Answer:

<code>concat()</code>	<code>merge()</code>
Stacks DataFrames	Joins on common columns
Axis-based (0 or 1)	Column-based
No common key needed	Requires common key

 Quick Revision Table

Topic	Key Functions
NumPy Creation	<code>array()</code> , <code>arange()</code> , <code>linspace()</code> , <code>zeros()</code> , <code>ones()</code>

Topic	Key Functions
NumPy Attributes	shape , size , dtype , ndim
NumPy Methods	min() , max() , sum() , mean() , std() , argmax()
NumPy Indexing	arr[0] , arr[1:5] , arr[arr>5]
NumPy Reshape	reshape() , resize()
Pandas Creation	Series() , DataFrame()
Pandas Selection	loc[] , iloc[] , df['col']
Pandas Missing	isna() , dropna() , fillna()
Pandas Merge	merge() , join() , concat()
Pandas Group	groupby() , agg()
Pandas Pivot	pivot_table() , crosstab()
Matplotlib	plot() , scatter() , hist() , subplot()
Seaborn Distribution	histplot() , jointplot() , pairplot()
Seaborn Categorical	countplot() , barplot() , boxplot() , violinplot()
Seaborn Matrix	heatmap() , clustermap()
Seaborn Regression	lmplot()
Plotly	ipplot()
Cufflinks	cf.go_offline()

End of Interview Questions & Answers

Good luck with your interviews! 🚀